

ISIDOR Research Summary

StepScore / CAD42 Platform — Relevant Extracts
Condensed from ISIDOR_RESEARCH_1.0.pdf (120 pages)

Contents

1. Project Overview & Architecture
2. Benchmark Harness Design
3. Prompt Writing & Taxonomy (L0-L3)
4. Part Families & Dataset Design
5. Alignment Pipeline (PCA + ICP)
6. The 44-Metric Engine
7. Threshold Definitions & Calibration
8. Threshold Tuning Methodology
9. ML-Powered Scoring System
10. Grading Profiles & Failure Taxonomy
11. Roadmap & Future Directions

1. Project Overview & Architecture

The ISIDOR/CAD42 project aims to build a rigorous, reproducible benchmark platform for evaluating AI-generated CAD models against ground-truth reference geometry. The core insight is that CAD evaluation requires deterministic geometric comparison, not subjective LLM-based judging.

Core Architecture Decision

The research evaluated three evaluation approaches and concluded that a hybrid deterministic-first pipeline is optimal:

- **Programmatic geometric checks (primary, 70%):** Deterministic metrics computed from mesh/point-cloud comparison. Fast, reproducible, auditable.
- **LLM qualitative judge (secondary, 30%):** For design-intent and feature-tree quality checks that pure geometry cannot capture.
- **Hybrid pipeline:** Fast geometric filter eliminates ~70% of obvious failures; LLM judge handles remaining edge cases.

The final architecture:

- STEP as canonical ground-truth artifact
- Tessellate both reference and generated STEP to STL/point-clouds with fixed settings
- Align meshes deterministically (centroid + PCA + ICP)
- Compute 44 geometric metrics on aligned meshes
- Apply per-metric thresholds for pass/fail gates
- Compute composite weighted score for overall quality ranking
- ML model layer for combined predictive scoring

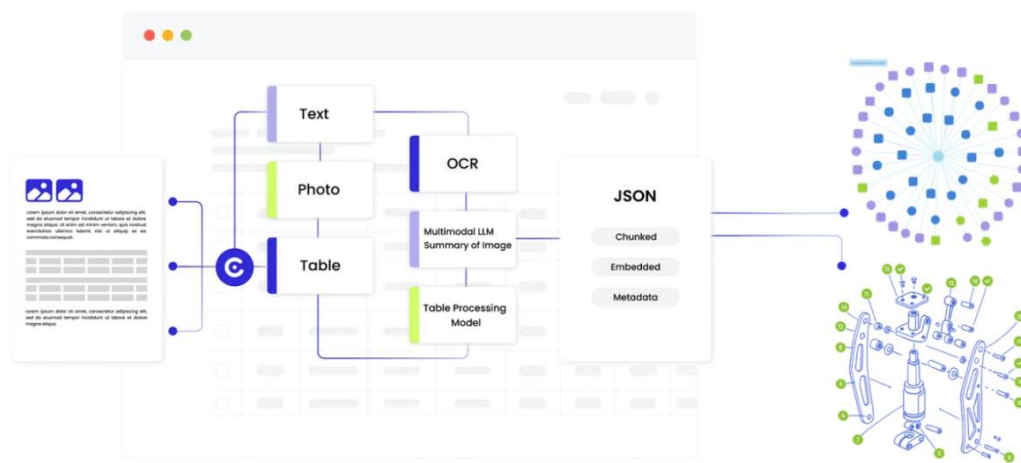


Fig 1. High-level CAD evaluation pipeline overview

Key Design Principles

- **Pairwise comparison over absolute scoring:** Compare generated vs reference, not score in isolation
- **Metric bundle, not single metric:** No single metric captures all failure modes
- **Calibrate thresholds statistically:** Use labeled positive/negative pairs to tune, not arbitrary cutoffs
- **Reproducibility controls:** Fixed tessellation, fixed random seeds, fixed sampling counts
- **Failure taxonomy:** Track generation failure vs conversion failure vs alignment failure vs metric failure

2. Benchmark Harness Design

The benchmark is structured as a two-track system to capture both precision and practical usefulness:

Deterministic Track

- Strict pass/fail against one reference STEP
- Prompts are fully specified with exact dimensions, coordinates, constraints
- Measures: can the model follow exact engineering specs?
- Useful for enterprise buyers, QA teams, safety/compliance use cases

Inference Track

- Prompts intentionally omit some non-critical details
- Model must make reasonable assumptions
- Looser scoring or multiple acceptable references
- Measures: practical usefulness when real prompts are incomplete

Harness CLI Architecture

The benchmark runs via a CLI harness that processes a manifest CSV of tasks:

- **Inputs per task:** task_id, prompt, reference_step, optional expected_bbox/components
- **Outputs per run:** generated STEP (or SCAD converted to STEP), metrics, pass/fail, logs
- **CLI command:** `run_benchmark --tasks ... --models ... --replicates 5`
- **Artifacts saved** under `results/<run_id>/`
- **Summary reports:** pass rate per model, mean/std of Chamfer/Hausdorff, conversion failure rate, cost/time

Task Schema (YAML)

Each benchmark task is defined by a structured YAML schema:

- **task_id:** Unique identifier (e.g., I_bracket_001)
- **description:** Canonical model prompt (min 10 chars)
- **reference_step:** Path to ground-truth STEP file
- **units:** mm/cm/m/in (default: mm)
- **difficulty:** easy/medium/hard
- **requirements:** bounding_box, topology_requirements (component count, watertight)
- **metric_thresholds_override:** Optional per-task threshold overrides

Reproducibility Controls

- Use identical prompt template across models
- Run at least 3 replicates per model-task pair
- Track mean and std for Chamfer and Hausdorff 95p
- Flag unstable tasks with high metric variance
- Fixed random seeds, same sampling count for point clouds

Sanity Validation Cases

- Identical mesh vs itself: expect near-zero distances
- Translated/rotated same mesh: should still pass after alignment
- Known bad geometry: should fail
- Track failure taxonomy: generation, conversion, alignment, metric failures

3. Prompt Writing & Taxonomy (L0-L3)

4-Level Prompt Taxonomy

The dataset uses four prompt difficulty/detail tiers to evaluate model robustness across different levels of specification:

Level	Name	Description	Example Detail
L0	Abstract	Coarse shape description only	'Create an L-bracket'
L1	Beginner	Simple procedural instructions	Mentions legs, approximate size
L2	Intermediate	More geometric detail	Explicit dimensions, hole positions
L3	Expert	Full parametric/relative-value instructions	Exact coords, tolerances, constraints

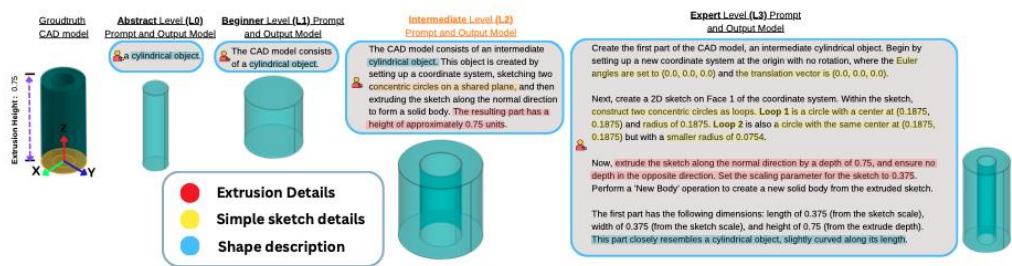


Figure 4: Parametric CAD model generation by Text2CAD transformer using different text prompts. Our text prompts follow a certain structure highlighting the different design aspects of CAD construction workflow (shown in different colors).

Fig 2. 4-Level prompt taxonomy visualization (from Text2CAD research)

Why Multiple Prompt Levels Matter

- **Measures prompt robustness:** A good CAD model should produce similar geometry regardless of prompt detail level
- **Maps to real-world usage:** Enterprise users write L3 prompts; casual users write L0-L1
- **Separates model capabilities:** Spec compliance (L3) vs geometric inference (L0-L1)
- **Each level tests different skills:** L3 tests precision adherence; L0 tests whether the model can infer missing details

Metadata-First Prompt Factory

Rather than manually writing prompts, the research recommends an automated pipeline:

- Ingest curated STEP files and normalize units/orientation (mm, right-handed frame)
- Auto-extract CAD metadata per part (OpenCascade/CadQuery): bbox, volume, surface types, hole counts, symmetry axes
- Store one canonical JSON record per part (ground-truth intent layer)
- Generate prompt variants from JSON using templates: L0, L1, L2, L3
- Use LLM only for paraphrasing/rewording, not for inventing geometry
- Run prompt linter + validator: required sections present, dimensions included, no contradictions
- Run automatic sanity replay: sample prompts to generate CAD, compare against reference

Required Prompt Structure

- **Part identity:** One line naming the part and objective
- **Units and frame:** mm, right-handed XYZ, explicit origin
- **Base geometry:** Primitive type, dimensions, exact placement

- **Additive features:** Bosses, ribs, extrusions with exact dimensions
- **Subtractive features:** Holes, slots, pockets with axis, center, depth
- **Constraints:** Symmetry, concentricity, equal spacing, tangency
- **Topology:** Single connected solid, no floating bodies

Prompt Level Examples — L-Bracket Renders

Below are example renders showing the same L-bracket part generated from different prompt levels, demonstrating how detail affects output quality:

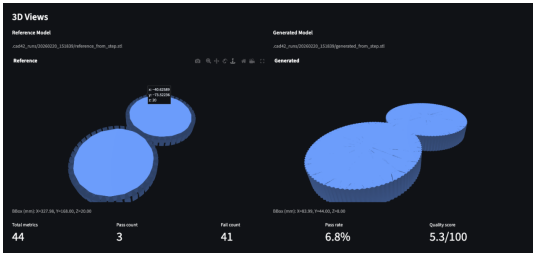


Fig 3a. L1, V1 — Beginner-level prompt, variant 1

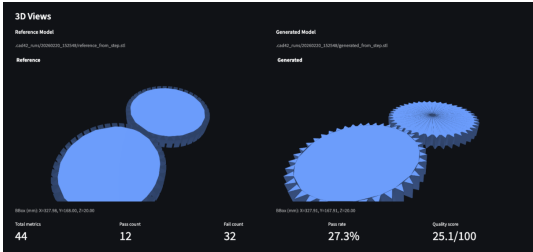


Fig 3b. L1, V2 — Beginner-level prompt, variant 2

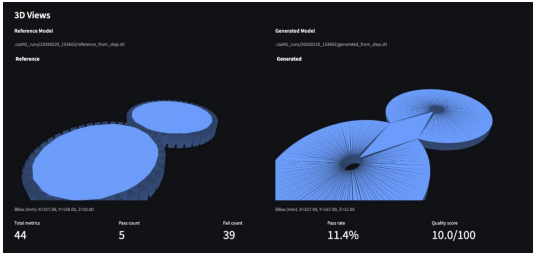


Fig 3c. L2, V1 — Intermediate-level prompt

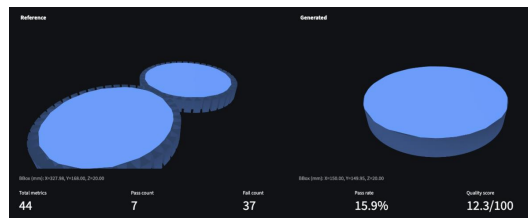


Fig 3d. L2, V3 — Intermediate-level, variant 3

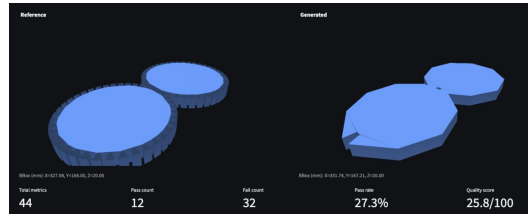


Fig 3e. L3, V2 — Expert-level prompt

Prompt Variants and Replicates

Each prompt level has multiple paraphrased variants (V1, V2, V3) to test model consistency. Replicate guidance calls for at least 3 replicates per model-task pair, tracking mean and standard deviation for key metrics.

4. Part Families & Dataset Design

The benchmark uses a curated set of part families spanning different geometric complexities and manufacturing feature types:

Basic Families (9 families)

Family	Key Features	Complexity
block_with_hole	Simple block + through-hole	Easy
flange	Circular base + bolt holes	Easy-Medium
I_bracket	Two perpendicular legs + holes/slots	Medium
stepped_shaft	Cylindrical steps + keyways	Medium
spur_gear	Involute teeth + hub	Hard
ring_spacer	Simple cylindrical ring	Easy
box_hole	Rectangular body + holes	Easy
slotted_plate	Flat plate + slots/cutouts	Easy-Medium
shell_enclosure	Hollow shell body	Medium-Hard

Extended Families (additional)

- **Spur gear (coarse/fine):** Tests rotational symmetry handling
- **Manifold block:** Complex internal channels
- **Keyed shaft + hub:** Assembly-like multi-feature part
- **Bearing housing:** Precision cylindrical features

Dataset Scale

- Target: 600-package manifest across all families
- Each package: task_id, reference_step, prompt variants (L0-L3), expected constraints, thresholds
- Pilot testing: 200 labeled pairs for initial threshold calibration
- Per-family minimum: 30 labeled pairs needed for per-family threshold tuning

Available Datasets for Reference STLs

Dataset	Size	License
ABC Dataset (Autodesk)	~1M models	Varies by source
Fusion 360 Gallery	~8K assemblies, ~154K parts	Non-commercial research
DeepCAD	~178K models	Research use
MFCAD	~60K models	Free access

5. Alignment Pipeline (PCA + ICP)

Alignment is a critical preprocessing step before metric computation. Without proper alignment, distance metrics reflect positional/rotational offset rather than actual geometric differences. The research establishes a multi-stage pipeline:

Stage 1: Centroid Translation

- Compute each shape's center of mass (centroid)
- Translate both shapes so centroids sit at the origin
- Removes all positional offset between the shapes

Stage 2: PCA Pre-Alignment

- Compute covariance matrix of centered point clouds
- Extract eigenvectors (principal axes) and eigenvalues (variance magnitudes)
- Eigenvectors define the canonical orientation of each shape
- Rotate one shape so its PCA axes align with the other's
- Removes most rotational offset before ICP refinement

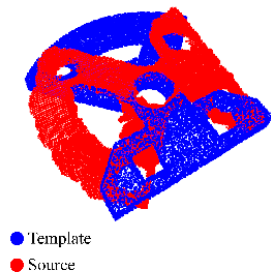


Figure 1. An example of a solution with a negative R^2 value. The estimated transformation (red) does not overlap the template (blue). The result shown is for the base-top plate object, after applying PointNetLK.

Fig 4. PCA + ICP alignment concept — rough alignment before ICP refinement

Stage 3: ICP Refinement

- Iterative Closest Point algorithm fine-tunes translation and rotation
- Requires decent starting pose (provided by PCA pre-alignment)
- Converges to local minimum; quality depends on initialization
- Multiple sign-flip trials (8 combinations of axis flips) to handle PCA sign ambiguity
- Best trial selected by lowest RMSE

Stage 4: Symmetry-Aware Alignment

Rotationally symmetric parts (gears, bearings, flanges) cause PCA eigenvalue degeneracy, leading to arbitrary in-plane rotation and corrupted metrics. The fix:

- **Detect eigenvalue degeneracy:** If ratio of consecutive eigenvalues < 1.5 , shape is symmetric
- **Identify symmetry axis:** Eigenvector with the distinct (non-degenerate) eigenvalue
- **Generate angular rotation trials:** 24 angles at 15-degree increments around symmetry axis
- **Combined trials:** 24 angles x 8 sign flips = 192 ICP initializations
- **Fallback:** Standard 8 trials when eigenvalues are well-separated (non-symmetric parts)

2.4. Ground-Truth Estimation

To compute the metrics from Section 2.2, we needed to estimate the ground truth. The process was based on using known information, like the object's orientation on the table, estimated values such as the normal vector on the table and finally, a visual correction for the translation [6]. We placed the 3D camera parallel to the table, which meant the x-axis of the template was too. Figure 5 shows the steps, which are detailed below.

1. We centred both point clouds on the origin by subtracting their mean.
2. Using an estimation of the normal vector on the table, we performed a rotation around the template x-axis, aligning the y-axes of both objects.
3. Given the known rotation of the object on the table, we performed a final rotation around the new y-axis.
4. We performed visual corrections for the rotation and mainly translation, similar to [6].

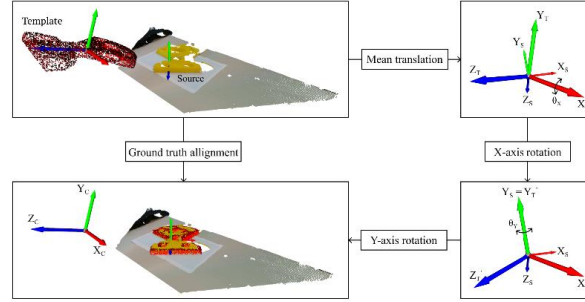


Figure 5. Steps performed to estimate the ground-truth estimation. The red and yellow point clouds represent the template and source, respectively. $X_T Y_T Z_T$ is the template's and $X_S Y_S Z_S$ the source's coordinate system.

Fig 5. Registration methods comparison from alignment research

Alignment Methods Available in StepScore

Method	Description	When to Use
pca_icp	PCA pre-align + 8 sign-flip trials + ICP (default)	General parts: boxes, brackets, shafts
pca_icp_symmetric	PCA + angular rotation trials around detected symmetry axis (up to 192 trials)	Gears, bearings, bolt-hole flanges
icp_only	Centroid translation + 8 sign-flip ICP, no PCA	Testing ICP without PCA influence
centroid_only	Translate centroids only, no rotation	Pre-oriented parts or debugging
none	No transform at all	Already-aligned parts or debugging

Alignment Quality Checks

- **Store ICP fitness** per run for debugging and thresholding
- **Fail alignment explicitly** if best ICP fitness below minimum (prevents distance metrics from reflecting alignment failure, not geometry error)
- **Registration failure rate** tracked as a benchmark diagnostic metric
- Record alignment_mode in results schema for apples-to-apples comparison

Research-Informed Alternatives

The research surveyed multiple registration methods and their tradeoffs:

- **RANSAC + ICP**: Best practical accuracy/speed baseline for batch benchmarking
- **GO-ICP**: Highest accuracy but slow; useful as high-confidence audit pass
- **TEASER++**: Robust global registration under extreme outliers
- **MRPCA-ICP**: Major-region PCA for partial/incomplete overlap cases
- **Trimmed ICP**: Better when overlap is partial or there are outliers
- **Feature-based (FPFH/SHOT)**: Good coarse initialization before ICP

6. The 44-Metric Engine

The metric engine computes 44 geometric metrics across 8 categories, ranked by diagnostic importance. Metrics are computed on aligned meshes/point-clouds after the alignment pipeline completes successfully.

Metric Categories & Priority Ranking

#	Metric	Purpose
1	Valid CAD rate	Hard gate: output parses/converts at all
2	Alignment quality (ICP fitness + RMSE)	Ensures distances are trustworthy
3	Chamfer distance	Average shape mismatch (primary)
4	Hausdorff 95p	Near-worst local errors
5	Volume difference %	Global mass/size correctness
6	Bounding-box error	Overall dimension sanity
7	Component count match	One part vs disconnected pieces
8	Watertight/manifold	Manufacturability prerequisite
9	Critical dimension error	Key engineering dimensions
10	Tolerance-band pass rate	Fraction of dims within tolerance

Full 44-Metric List (11-44)

#	Metric	#	Metric
11	Hausdorff 99p	28	Cross-section IoU
12	Point-to-surface stats	29	Slice contour distance
13	Feature/primitive count	30	Voxel IoU / Dice
14	Self-intersection count	31	Occupancy precision/recall/F1
15	Generation success rate	32	EMD/Wasserstein distance
16	Replicate consistency	33	Curvature distribution
17	Pass@k	34	Feature-edge overlap
18	Runtime	35	Signed volume difference
19	Cost per sample	36	Silhouette IoU
20	Oriented bbox error	37	SSIM/LPIPS on renders
21	Surface area diff %	38	Registration failure rate
22	Normal consistency	39	Perturbation reliability
23	Mass properties (COM/inertia)	40	Composite weighted score
24	Centroid offset	41	Edge chamfer distance
25	Inertia tensor diff	42	Normal angle error
26	Euler characteristic/genus	43	Point-to-surface max
27	Hole/void count match	44	Hole count match

Key Metric Definitions

- **Chamfer distance:** Average nearest-point distance between sampled surfaces. Primary geometry similarity metric.
- **Hausdorff distance:** Maximum nearest-point distance (95p/99p percentiles for robustness). Captures worst-case local errors.
- **Edge chamfer:** Average nearest distance between edge points. Characterizes contour structure fidelity.
- **Normal angle error:** Angular consistency between predicted and true surface normals.
- **Voxel IoU:** Intersection over Union on voxelized occupancy grids. Resolution-dependent.
- **Cross-section IoU:** Profile overlap in critical slice planes. Strong for profile-controlled geometry.
- **EMD/Wasserstein:** Earth Mover's Distance — global shape transport metric. Strict but expensive.
- **Silhouette IoU:** Visual profile consistency across fixed view angles.

7. Threshold Definitions & Calibration

Initial threshold priors for a mechanical CAD benchmark in mm units. These are starting points for calibration, not final production values:

#	Metric	Threshold	Rationale
1	valid_cad_rate	≥ 1.00	Hard gate: invalid CAD unusable
2	icp_fitness	≥ 0.30	Below this, alignment unreliable
3	alignment_rmse_mm	≤ 1.00	Registration within tolerance
4	chamfer_mm	≤ 1.00	Global similarity threshold
5	edge_chamfer_mm	≤ 0.80	Edge fidelity stricter than surface
6	hausdorff_95p_mm	≤ 1.50	Robust near-worst-case
7	hausdorff_99p_mm	≤ 2.50	Allows small outlier spikes
8	volume_diff_%	≤ 2.00	Engineering-level volume tolerance
9	surface_area_%	≤ 3.00	Area noisier than volume
10	bbox_error_mm	≤ 1.00	Overall size close to reference
11	component_count	$= 1$	Hard topology gate
12	watertight_manifold	$= 1$	Manufacturing prerequisite
13	self_intersections	$= 0$	Invalid geometry flag
14	normal_consistency	≥ 0.95	Surface orientation quality
15	voxel_iou	≥ 0.88	Volumetric overlap quality
16	cross_section_iou	≥ 0.90	Profile overlap in key slices
17	silhouette_iou	≥ 0.92	Visual profile consistency
18	composite_score	≥ 0.85	Final acceptance gate

Important Calibration Notes

- These are **initial priors** — final thresholds must be calibrated using labeled positive/negative pairs
- For **deterministic track**: tighten by ~10-20%
- For **inference track**: loosen by ~10-25%
- Per-family thresholds recommended when sufficient labeled data exists (30+ pairs per family)
- Model choice can bias tuning — do not tune from one model only

8. Threshold Tuning Methodology

The research establishes a rigorous threshold tuning methodology using labeled pairs and cost-sensitive optimization:

Cost-Sensitive Threshold Optimization

- Choose threshold that minimizes a cost function: **Cost = FP_cost x FP + FN_cost x FN**
- Default asymmetric costs: FP_cost = 2.0, FN_cost = 1.0 (false passes more expensive than false fails)
- Sweep all candidate thresholds per metric, compute cost at each, select minimum
- Track objective metric (F1, accuracy, or weighted cost) for each threshold candidate

Tuning Example

With 30 labeled pairs (15 positive, 15 negative) and FP cost=2, FN cost=1:

- Threshold A: FP=2, FN=5 => Cost = 2x2 + 1x5 = 9
- Threshold B: FP=6, FN=3 => Cost = 2x6 + 1x3 = 15
- Threshold A wins despite more FN because FP are costlier

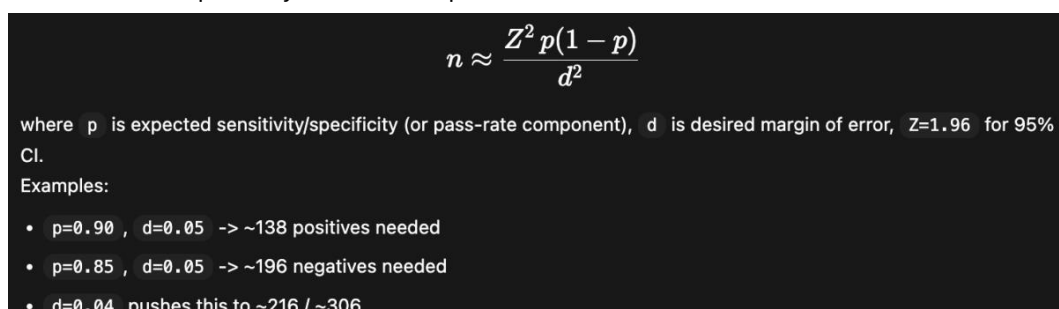
Per-Family Threshold Tuning

Different part families may need different thresholds due to varying geometric complexity. The `auto_tune_thresholds_by_family.py` script:

- Reads labeled pairs CSV with a 'family' column
- Splits data by family, creates per-family pair subsets
- Requires minimum 30 rows per family (configurable via `--min-family-rows`)
- Runs threshold optimization independently for each qualifying family
- Produces `thresholds_by_family.json` with per-family overrides
- Skips families with too few rows or no threshold improvements found

Labeling Requirements

- Pilot testing: 200 labeled pairs recommended
- Label categories: positive (usable/pass) vs negative (unusable/fail)
- Need both positive and negative labels per family
- Inter-rater reliability: human spot-checks on 5-10% of outputs
- Calibration set should span easy/medium/hard parts


$$n \approx \frac{Z^2 p(1-p)}{d^2}$$

where `p` is expected sensitivity/specificity (or pass-rate component), `d` is desired margin of error, `Z=1.96` for 95% CI.

Examples:

- `p=0.90` , `d=0.05` -> ~138 positives needed
- `p=0.85` , `d=0.05` -> ~196 negatives needed
- `d=0.04` pushes this to ~216 / ~306

Fig 6. Labeling and threshold tuning workflow overview

9. ML-Powered Scoring System

The research describes a two-layer scoring architecture that combines interpretable per-metric thresholds with a predictive ML model:

Two-Layer Architecture

Layer	What it Does	Output
Layer 1: Per-Metric Thresholds	44 individual pass/fail checks. Diagnostic — tells WHAT failed and WHY. Interpretable, auditable.	44 pass/fail flags + quality_score_0_100
Layer 2: ML Combined Score	Single model sees all 44 metrics at once. Predictive — captures metric interactions. Tells HOW GOOD overall.	Probability 0.0-1.0

Why ML Over Pure Thresholds

- **Combined score is more accurate:** A part can fail one metric marginally but be excellent on others — threshold-only misses this
- **Captures metric interactions:** Some metrics compensate for others (e.g., high chamfer but perfect topology may still be usable)
- **Weighted confidence:** ML learns which metrics matter most, rather than equal weighting
- **Calibrated probability:** Output of 0.82 genuinely means ~82% chance a human expert would say 'usable'

ML Model Pipeline

- **Features (X):** 44 metric columns from labeled_pairs_for_review.xlsx
- **Labels (y):** Binary positive/negative from human review
- **Handle NaN/inf:** Fill missing with worst possible value (failure signal)
- **Split:** 80% working set, 20% blind test (stratified by label)
- **Working set:** Further split into calibration (train) and validation

Model Options (Ranked by Recommendation)

Model	Pros	Cons
Logistic Regression	Interpretable, fast, good baseline. Gets ~90% of the way.	Misses non-linear metric interactions
Gradient Boosted Trees (XGBoost/LightGBM)	Handles interactions, feature importance, top accuracy	Less interpretable, needs more data
Random Forest	Robust, handles non-linearity	Can overfit on small datasets

Calibration and Deployment

- Use Platt scaling or isotonic regression for probability calibration
- Save trained model + scaler + threshold as a versioned artifact (model_card.json)
- Model card records: training date, feature list, performance metrics, threshold
- Score output: `stepscore_ml = 0.82` means 82% confidence of usability
- Compare ML score against threshold-only score to measure improvement

10. Grading Profiles & Failure Taxonomy

Grading Profiles

StepScore supports multiple grading profiles that weight the 44 metrics differently based on use case:

- **full_44**: All 44 metrics with balanced weights (default)
- **geometry_only**: Distance metrics only (Chamfer, Hausdorff, volume, bbox)
- **manufacturing_ready**: Emphasizes watertight, self-intersections, topology
- **visual_quality**: Emphasizes silhouette IoU, render SSIM/LPIPS, normal consistency
- **quick_check**: Reduced subset for fast pass/fail screening

Failure Taxonomy

Tracking failure modes is critical for diagnosing pipeline issues vs model issues:

- **Generation failure**: Model fails to produce any output (timeout, error, refusal)
- **Conversion failure**: Output cannot be converted to mesh (invalid STEP/SCAD)
- **Alignment failure**: ICP cannot find acceptable alignment (fitness below threshold)
- **Metric failure**: Metrics computed but fall outside thresholds
- **Partial failure**: Some metrics pass, others fail — captured by per-metric flags

Per-Operation Analysis

Report pass rates broken down by manufacturing operation type:

- Extrusion operations: pass rate, mean error
- Hole operations: pass rate, diameter accuracy
- Fillet/chamfer operations: detection rate, radius accuracy
- Boolean operations: topology correctness rate
- Pattern operations: count accuracy, spacing accuracy

Reporting Dimensions

- Per-model performance: overall pass rate, mean scores by family
- Per-family performance: which families each model handles well/poorly
- Per-difficulty tier: Easy/Medium/Hard pass rates (e.g., 95% Easy, 72% Medium, 31% Hard)
- Per-prompt-level: L0 vs L1 vs L2 vs L3 performance delta
- Per-operation: which CAD operations cause most failures
- Confidence intervals: mean +/- std over replicates, flag high-variance tasks

11. Roadmap & Future Directions

Near-Term (Implemented or In Progress)

- 44-metric engine with configurable grading profiles
- PCA + ICP alignment with 5 selectable methods (including symmetry-aware)
- CLI tool (stepscore_cli.py) with score and run subcommands
- Streamlit dashboard (app.py) for visual metric exploration
- Benchmark harness (harness_runner.py) for batch evaluation
- Per-family threshold tuning pipeline
- Cost-sensitive threshold optimization with configurable FP/FN costs

Medium-Term Priorities

- Scale labeled dataset to 200+ pairs for robust ML model training
- Train and deploy ML combined scoring model (logistic regression baseline)
- Expand part family coverage to 15+ families with 30+ pairs each
- Add inference track with looser thresholds for underspecified prompts
- Build automated prompt factory (metadata-first, template-based, LLM-paraphrased)
- Per-difficulty-tier threshold profiles

Long-Term Vision

- Industry-specific benchmark packs (automotive, aerospace, consumer products)
- GD&T-based evaluation (circularity, cylindricity, runout for precision parts)
- Assembly-level benchmarking (multi-part fit, clearance, interference checks)
- Drawing-to-STEP pipeline integration
- Continuous benchmark leaderboard for AI CAD model comparison
- B2B API for third-party model evaluation
- SO(3)-equivariant descriptors for alignment-free metrics (research direction)

Key Research References

- Text2CAD (2024): First text-to-parametric-CAD framework, 170K models, 660K text annotations
- CadEval: End-to-end CAD synthesis evaluation framework
- MRPCA-ICP (Liu et al., Optik 2021): Major-region PCA for partial overlap registration
- GO-ICP: Globally optimal ICP for high-accuracy alignment benchmark
- RANSAC + ICP: Best practical accuracy/speed tradeoff for batch benchmarking
- Graph Neural Networks for CAD: Feature recognition via B-Rep graph representation

Appendix A: STEP vs STL & Pipeline Performance

STEP vs STL: Roles in the Pipeline

Aspect	STEP	STL
Geometry type	Exact B-Rep (surfaces/solids)	Triangle mesh approximation
Use in pipeline	Source-of-truth artifact, topology checks, feature extraction	Distance metrics (Chamfer/Hausdorff), rendering, point sampling
Engineering checks	Volume, topology, manufacturability, parametric access	Limited to mesh-level properties
Interoperability	CAD-native interchange standard (ISO 10303)	Universal for rendering/printing

Best Practice

- Use STEP as source-of-truth artifact — keep it alongside any generated mesh
- Tessellate to STL/point-clouds for distance metrics with **fixed, deterministic settings**
- Use same mesher and same settings for both reference and generated models
- Normalize units before comparison (default: mm)
- Store tessellation parameters in run metadata for reproducibility

Memory & Performance Considerations

The metric engine can hit memory limits on complex parts. Key mitigations:

- **Fast mode:** Cap sample points to 4,000 and voxel pitch to $\geq 4.0\text{mm}$
- **Skip heavy metrics:** Disable voxel occupancy if estimated grid exceeds memory budget
- **Reduce EMD samples:** Earth Mover's Distance is $O(n^3)$ — cap sample size in fast mode
- **OOM protection:** Container killed with exit code 137 indicates out-of-memory; disable auto-restart to make crashes visible
- **Batch processing:** Harness runner processes tasks sequentially to limit peak memory

Tessellation Determinism

Tessellation non-determinism between separate STEP-to-STL conversions is a known issue. Two conversions of the same STEP file can produce slightly different triangle meshes, causing non-zero distance metrics even for identical models. Mitigations:

- Use the same tessellation library and version across all conversions
- Fix linear and angular deflection parameters
- Pre-tessellate reference models and store alongside STEP
- Set voxel pitch high enough to absorb minor tessellation noise

Appendix B: Gear & Symmetric Part Examples

Rotationally symmetric parts are the most challenging category for the alignment pipeline. Below are examples from the research showing gear and part renders used for testing:

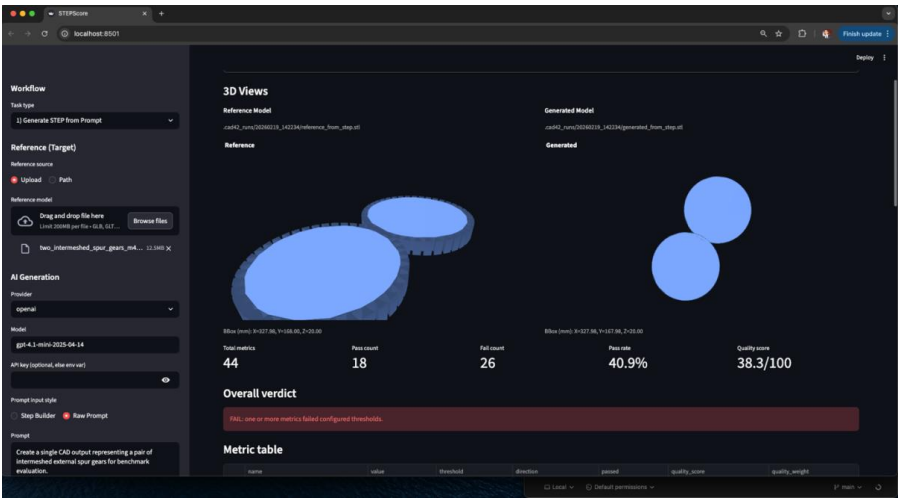


Fig 7a. Gear test renders — additive features only

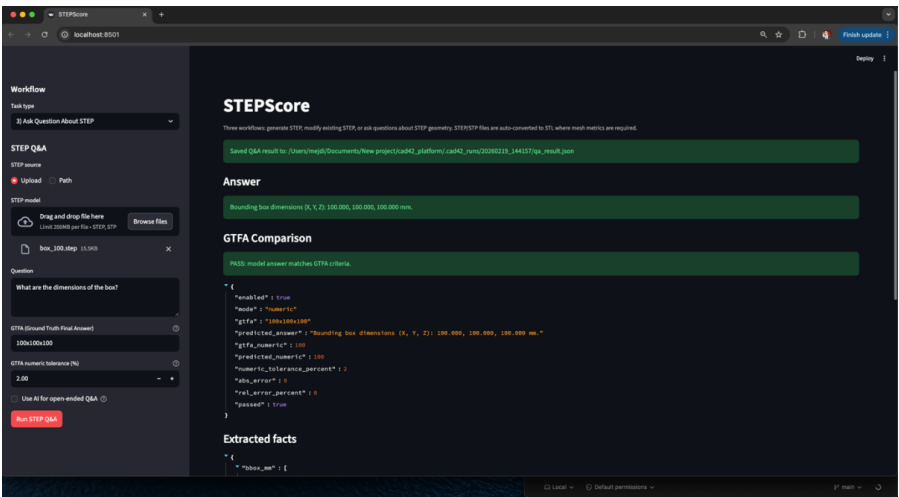


Fig 7b. Gear test renders — reference vs generated comparison

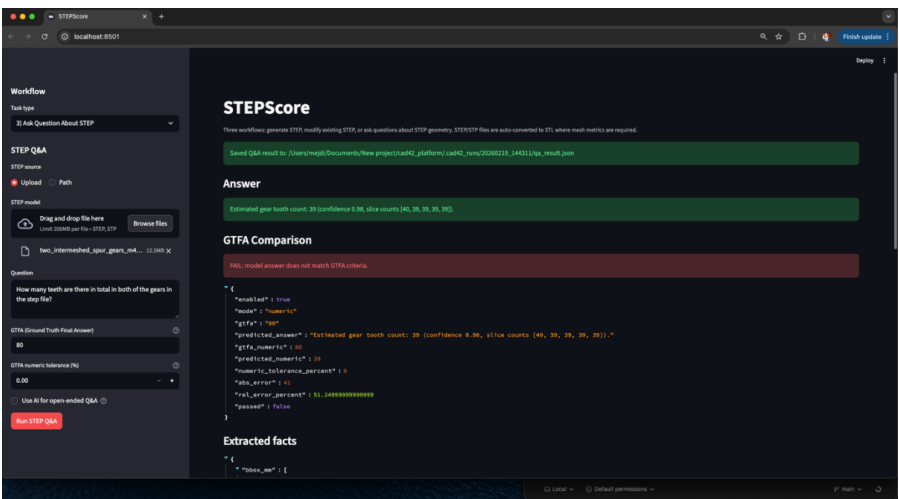


Fig 7c. Test results showing key fixes needed for symmetric part alignment

Design Constraint Renders

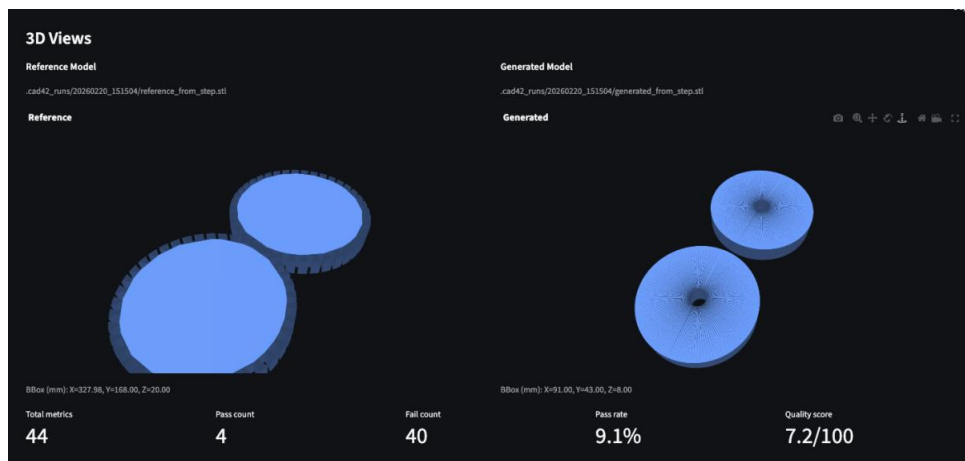


Fig 8. Design constraint visualization for benchmark part specifications

Appendix C: External Research & CadEval

The ISIDOR research surveyed several external frameworks and datasets that informed the StepScore design:

CadEval Framework



Fig 9. CadEval framework — end-to-end CAD synthesis evaluation

- Measures end-to-end CAD synthesis: text prompt to parametric CAD model
- Supports standard LLMs via API
- Evaluates both geometric and parametric correspondence
- Key insight adopted: need standardized benchmark, not ad-hoc evaluation

Deep Learning for CAD Survey

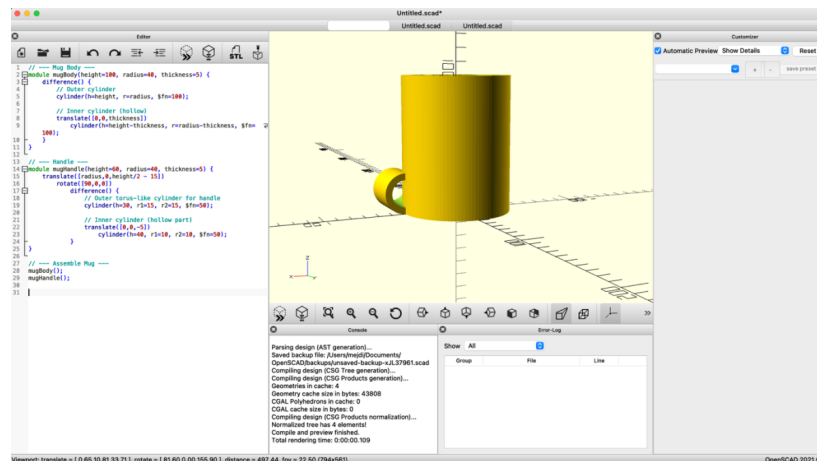


Fig 10. Survey of deep learning methods for 3D CAD reconstruction

Registration Method Comparison

Table 1. Parameters checked for each method. A check mark (✓) or range indicates the parameter is verified, a dash (/) means the parameter is not applicable and a lightning symbol (!) indicates the parameter is checked, but no convergence was reached. Training models are reported in Section 2.3.

Method	GO-ICP	RANSAC	FGR	PointNetLK	RPMNet	ROPNet
Zero mean	✓	✓	✓	✓	✓	✓
Refinement	✓	✓	✓	✓	✓	✓
Bounding box	!	1 → 1.8				/
Voxel size [m]	$10^{-3}, 10^{-2}$	$10^{-4} \rightarrow 10^{-1}$		$0, 10^{-3} \rightarrow 10^{-2}$		
MSE Threshold [m]	$10^{-5} \rightarrow 10^{-1}$	/	/	/	/	/
Trim fraction [/]	$10^{-4} \rightarrow 10^{-1}$	/	/	/	/	/
Training model	/	/	/	✓	✓	✓

We adapted the bounding box to simulate the effect of different cutouts around the object, as in Figure 6. The larger the bounding box, the more environmental information is included in the point cloud. However, methods that can work for larger bounding boxes require less pre-processing. This is especially interesting in real-time applications. Furthermore, the maximal bounding box places additional requirements on preliminary object detection and filtering steps.

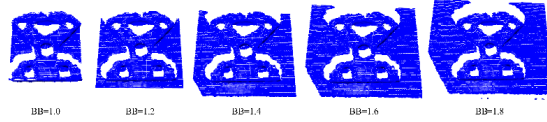


Figure 6. Effect of adapting the bounding box to increase the information around the object in the point cloud

Fig 11. Comparison of 6 registration methods for CAD-to-scan alignment

- GO-ICP: Highest accuracy, slowest runtime
- RANSAC + ICP: Best accuracy-speed tradeoff (adopted as our default approach)
- FGR: Fast but less reliable on noisy/harder cases
- Learning-based (PointNetLK, RPMNet): Fast but domain-dependent generalization